AASTool PDF Generation

AASTool — PDF Generation Subsystem

Enterprise Edition | Version 1.0.0

Table of Contents

Overview

The PDF generation subsystem produces formal accessibility certification reports from evaluation results. Reports are generated server-side from the Express backend, combining evaluation data from the database with branded templates to produce EN 17210-aligned certification documents.

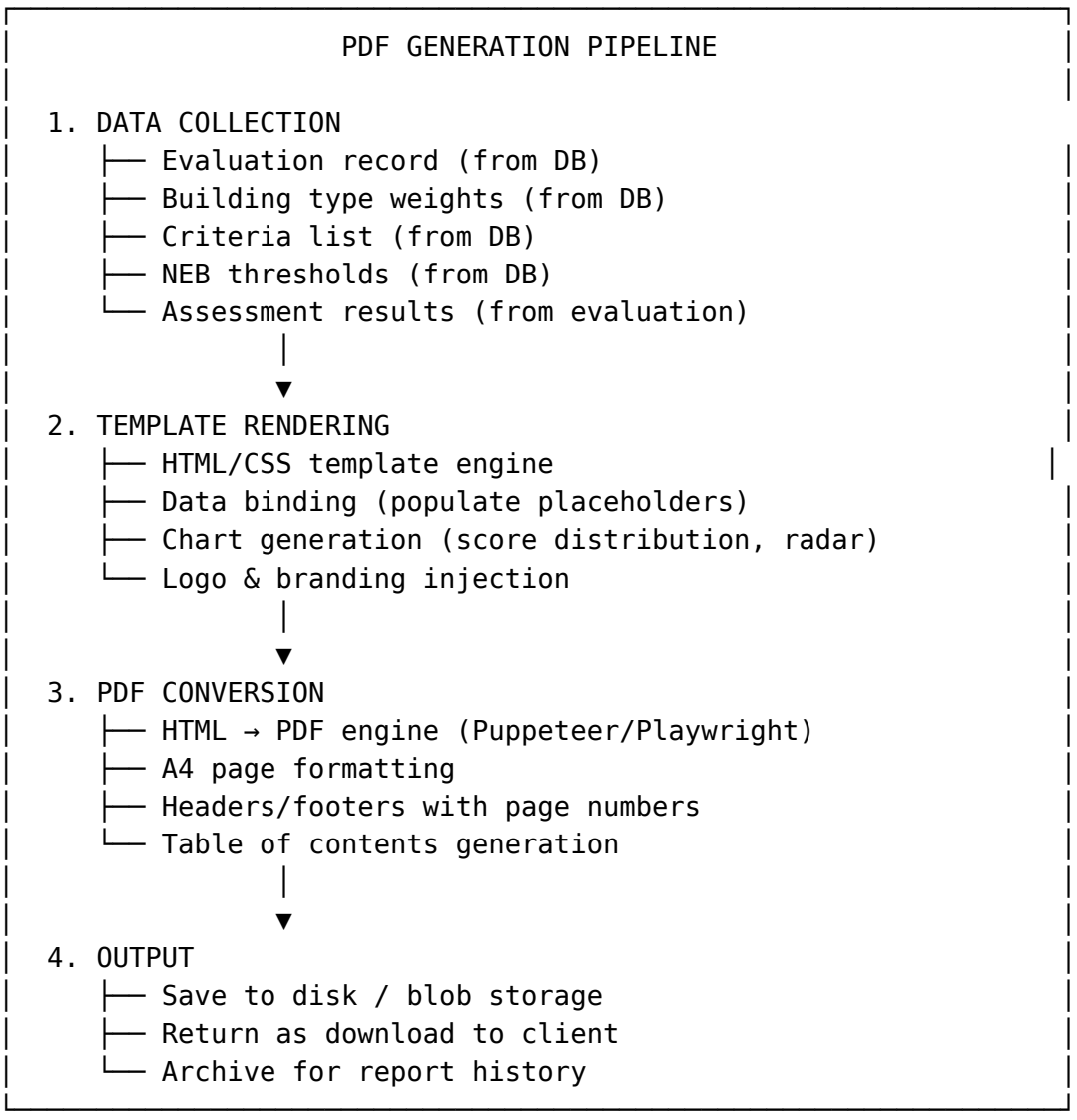
Key Requirements

- **Server-side generation** — PDFs are produced by the backend, not the browser

- **Data-driven** — All scores, classifications, and criteria come from the database
- **Branded** — Reports carry the AASTool/SERG brand identity
- **Structured** — Consistent sections: header, building info, scores, NEB classification, recommendations, appendices
- **Printable** — A4 format, print-ready with proper margins and page breaks
- **Accessible** — Tagged PDF structure for screen reader compatibility

---

Architecture



Report Types

1. Full Certification Report (~30 pages)

The complete accessibility audit report, suitable for formal certification submission.

Sections: 1. **Cover Page** — Title, building name, date, NEB class badge, AASTool branding 2. **Executive Summary** — OBS score, NEB classification, key findings (1 page) 3. **Building Profile** — Building type, DT weights, AD weights, assessment metadata 4. **Methodology** — EN 17210 alignment, scoring scale, weight derivation 5. **Overall Results** — OBS, NEB class, score distribution chart, average scores 6. **DT Analysis** — Per disability type: TIS scores, average scores, strengths/weaknesses 7. **AD Analysis** — Per assessment dimension: CIS scores, average scores, recommendations 8. **Strengths & Weaknesses** — Top 5 strengths, top 5 weaknesses with details 9. **Criterion Details** — Full 63-criterion table with scores, IS values, level assignments 10. **Recommendations** — Prioritized improvement actions by impact 11. **Appendices** — Raw data tables, methodology references, glossary

2. Summary Report (~5 pages)

A condensed version for stakeholders who need key metrics only.

Sections: 1. Cover info 2. OBS & NEB classification 3. Score distribution chart 4. Top 5 strengths and weaknesses 5. Key recommendations

3. Criteria Workbook (~15 pages)

The full criteria catalog with definitions and levels — used for scoring workshops.

Sections: 1. DT/AD matrix overview 2. Complete criteria table (code, name, definition, levels 1–5) 3. Scoring guide

---

PDF Generation Pipeline

Technology Stack

Component	Technology	Purpose
Template engine	Handlebars / EJS	HTML template with data binding

Component	Technology	Purpose
HTML→PDF converter	Puppeteer	Headless Chrome rendering to PDF
Charts	Chart.js (node-canvas)	Score distribution, radar charts
Fonts	Google Fonts (Inter, Roboto)	Brand typography in PDF
Storage	Filesystem / Azure Blob	Generated PDF storage

Alternative: Markdown → PDF

For documentation-only PDFs (e.g., this documentation package), use Pandoc:

```
pandoc docs/*.md -o AASTool-Documentation.pdf \
  --pdf-engine=xelatex \
  --toc --toc-depth=3 \
  --template=eisvogel \
  --metadata title="AASTool Enterprise Documentation"
```

Template Structure

HTML Template (Handlebars)

```
templates/
├── report-full.hbs          # Full certification report
├── report-summary.hbs       # Summary report
├── report-criteria.hbs      # Criteria workbook
├── partials/
│   ├── header.hbs           # Page header with logo
│   ├── footer.hbs           # Page footer with numbers
│   ├── cover.hbs            # Cover page
│   ├── neb-badge.hbs        # NEB classification badge
│   ├── score-chart.hbs      # Score distribution chart
│   ├── criteria-table.hbs   # Full criteria table
│   └── recommendations.hbs  # Recommendations section
├── styles/
│   └── report.css           # Print-specific CSS
└── assets/
    ├── logo.png             # AASTool logo
    ├── serg-logo.png        # SERG logo
    └── fonts/               # Embedded fonts
```

Data Binding

The template receives a context object:

```
        interface ReportContext {
          // Building info
          building: {
            id: string;
            name: string;
            type: string;
            disability_weights: number[];
            dimension_weights: number[];
          };

          // Evaluation results
          evaluation: {
            id: string;
            obs: number;
            nebClass: string;
            nebEquivalent: string;
            nebMeaning: string;
            nebScore: number;
            averageRawScore: number;
            avgRawScoreByDT: Record<number, number>;
            avgRawScoreByAD: Record<number, number>;
            scoreDistribution: Record<number, number>;
            strengths: CriterionResult[];
            weaknesses: CriterionResult[];
            tisByDT: Record<number, number>;
            cisByAD: Record<number, number>;
            criteria: CriterionResult[];
          };

          // Metadata
          metadata: {
            generatedAt: string;
            reportNumber: number;
            version: string;
            assessmentDate: string;
            assessorName?: string;
          };

          // Reference data
          reference: {
            dtLabels: { id: number; name: string }[];
            adLabels: { id: number; name: string }[];
            nebThresholds: { min: number; neb_class: string;
equivalent: string }[];
          };
        }
```

Data Sources

From Database

Data	Source Table	Query
Evaluation results	`evaluations`	`findOne({ where: { id } })`
Building type weights	`building_types`	`findOne({ where: { name } })`
Criteria definitions	`criteria`	`find()`
NEB thresholds	`neb_thresholds`	`find({ order: { min: 'DESC' } })`
Config	`config`	`find()`

From Calculation Engine

The `evaluate()` function already returns all data needed for PDF generation: - `obs`, `nebClass`, `nebEquivalent`, `nebMeaning`, `nebScore` - `averageRawScore`, `avgRawScoreByDT`, `avgRawScoreByAD` - `scoreDistribution` - `strengths[]`, `weaknesses[]` - `tisByDT`, `cisByAD` - `criteria[]` with computed `is` values

From API

The frontend can also trigger PDF generation via API:

```
POST /api/v1/reports/generate
Body: { evaluationId: "uuid" }
Response: PDF binary (Content-Type: application/pdf)
```

Implementation Guide

Backend Endpoint

```
// reportsController.ts — add to existing controller

import puppeteer from 'puppeteer';
import Handlebars from 'handlebars';
import fs from 'fs/promises';
```

```
        export async function generateReportPdf(req: Request, res:
Response) {
          try {
            const { evaluationId } = req.params;

            // 1. Load data
            const repo = AppDataSource.getRepository(Evaluation);
            const ev = await repo.findOne({ where: { id:
evaluationId } });
            if (!ev) return res.status(404).json({ ok: false,
error: 'Evaluation not found' });

            const bt = await
metadataService.getBuildingType(ev.building_type);
            const nebThresholds = await
metadataService.getNebThresholds();
            const dtLabels = await getDisabilityTypeLabels();
            const adLabels = await getAssessmentDimensionLabels();

            // 2. Build context
            const context: ReportContext = {
              building: {
                id: bt.id,
                name: bt.name,
                type: bt.name,
                disability_weights: bt.disability_weights,
                dimension_weights: bt.dimension_weights,
              },
              evaluation: ev.result,
              metadata: {
                generatedAt: new Date().toISOString(),
                reportNumber: await getNextReportNumber(),
                version: '1.0.0',
                assessmentDate: ev.created_at.toISOString(),
              },
              reference: {
                dtLabels,
                adLabels,
                nebThresholds,
              },
            };

            // 3. Render HTML
            const template = await fs.readFile('templates/report-
full.hbs', 'utf-8');
            const compiled = Handlebars.compile(template);
            const html = compiled(context);

            // 4. Convert to PDF
            const browser = await puppeteer.launch({ headless: true
});
```

```
            const page = await browser.newPage();
            await page.setContent(html, { waitUntil: 'networkidle0'
});
            const pdf = await page.pdf({
              format: 'A4',
              printBackground: true,
              margin: { top: '20mm', bottom: '20mm', left: '15mm',
right: '15mm' },
              displayHeaderFooter: true,
              headerTemplate: '<div style="font-size:8px;text-
align:center;width:100%">AASTool Certification Report</div>',
              footerTemplate: '<div style="font-size:8px;text-
align:center;width:100%">Page <span class="pageNumber"></span> of
<span class="totalPages"></span></div>',
            });
            await browser.close();

            // 5. Return PDF
            res.setHeader('Content-Type', 'application/pdf');
            res.setHeader('Content-Disposition', `attachment;
filename="AASTool-Report-${evaluationId}.pdf"`);
            res.send(pdf);
          } catch (err) {
            console.error('PDF generation error:', err);
            res.status(500).json({ ok: false, error: 'PDF
generation failed' });
          }
        }
```

Register Route

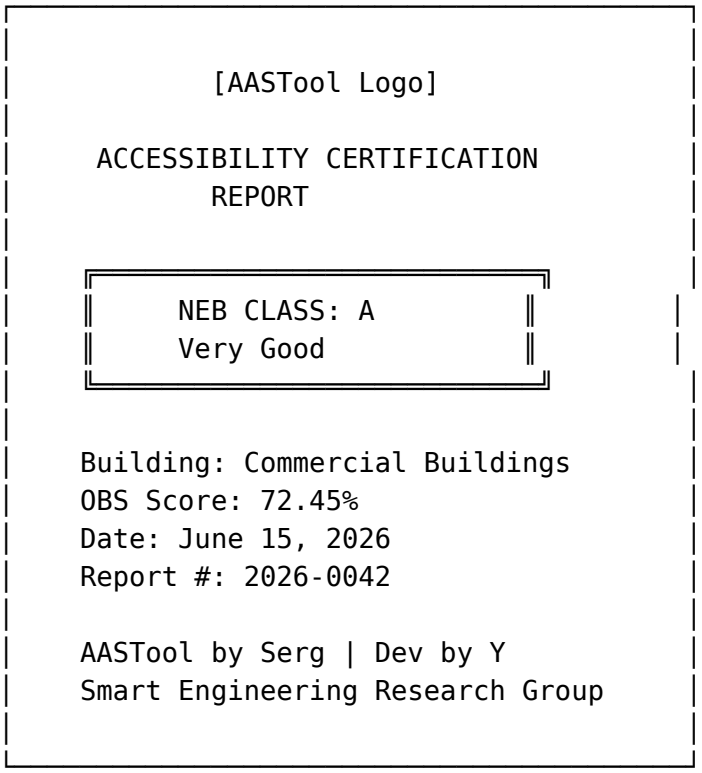
```
        // In index.ts
        app.get('/api/v1/reports/:evaluationId/pdf',
generateReportPdf);
```

Frontend Integration

```
        // lib/api/endpoints.ts — add
        downloadReportPdf: (evaluationId: string) => {
          const url = `${baseURL}/reports/${evaluationId}/pdf`;
          window.open(url, '_blank');
        },
```

NEB Certification Output

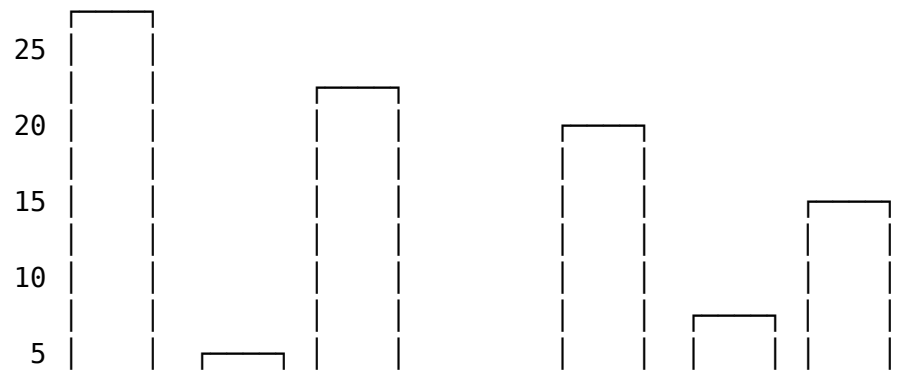
Cover Page Layout

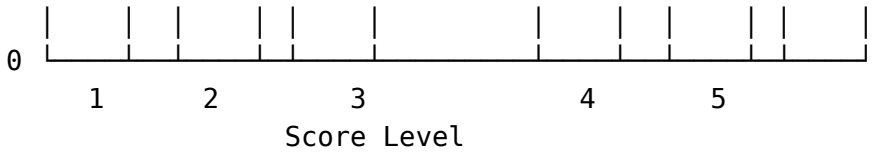


Results Table

DT	Disability Type	Avg Score	TIS	Weight
DT1	Physical	4.1	85.50	1.0
DT2	Sensory	3.5	72.30	0.8
DT3	Cognitive	3.9	81.00	1.2
DT4	Intellectual	3.8	79.20	1.0
DT5	Psychosocial	3.8	79.20	0.9

Score Distribution Chart





Styling & Branding

Print CSS Specifications

```css
/* report.css — Print-specific styles */

@page {
  size: A4;
  margin: 20mm 15mm;
  @bottom-center {
    content: "Page " counter(page) " of " counter(pages);
    font-size: 8pt;
    color: #666;
  }
}

body {
  font-family: 'Inter', 'Roboto', sans-serif;
  font-size: 10pt;
  line-height: 1.6;
  color: #1a1a1a;
}

.page-break {
  page-break-before: always;
}

.cover-page {
  display: flex;
  flex-direction: column;
  justify-content: center;
  align-items: center;
  height: 100vh;
  text-align: center;
}

.neb-badge {
  border: 3px solid #2563eb;
  border-radius: 12px;
  padding: 20px 40px;
  margin: 30px 0;
}
```

```
.neb-class {
  font-size: 36pt;
  font-weight: 800;
  color: #2563eb;
}

table {
  width: 100%;
  border-collapse: collapse;
  margin: 12px 0;
}

th {
  background: #f3f4f6;
  font-weight: 600;
  text-align: left;
  padding: 8px;
}

td {
  padding: 6px 8px;
  border-bottom: 1px solid #e5e7eb;
}

.strength-row { border-left: 3px solid #16a34a; }
.weakness-row { border-left: 3px solid #dc2626; }
```

Color Palette

Element	Color	Hex
Primary (NEB badges, headings)	Blue	`#2563eb`
Strengths	Green	`#16a34a`
Weaknesses	Red	`#dc2626`
Neutral (tables, borders)	Gray	`#e5e7eb`
Background	White	`#ffffff`
Text	Near-black	`#1a1a1a`

Rendering Pipeline

Dependency Installation

```
cd backend
npm install puppeteer handlebars
```

Puppeteer downloads a Chromium binary (~300MB). For production (Azure App Service), use:

```
npm install puppeteer-core @sparticuz/chromium
```

Azure App Service Configuration

For PDF generation in the Azure Linux App Service (restricted environment):

```
import puppeteer from 'puppeteer-core';
import chromium from '@sparticuz/chromium';

const browser = await puppeteer.launch({
  args: chromium.args,
  executablePath: await chromium.executablePath(),
  headless: chromium.headless,
});
```

Memory Management

- Close browser instances after PDF generation
- Use a browser pool for concurrent requests (max 3 instances)
- Set timeouts: 30 seconds for HTML render, 60 seconds total
- Clean up temp files in `/tmp` after generation

Performance Considerations

Concern	Solution
Large HTML (63 criteria × detail rows)	Lazy render sections, use CSS page breaks
Chart rendering	Pre-render charts as PNG with node-canvas, embed as base64
Browser memory	Pool browsers, close after use, max 3 concurrent
Response time	Generate async, poll for completion, email when ready
File size	Compress with `--compress` flag, target < 5MB
Font embedding	Subset fonts to only used glyphs using `subset-font`

Async Generation Pattern

For reports that take >5 seconds to generate:

```
POST /api/v1/reports/generate-async
→ { ok: true, jobId: "uuid" }

GET /api/v1/reports/jobs/:jobId
→ { status: "processing" } or { status: "done", downloadUrl: "..." }
```

Testing PDF Output

Manual Verification Checklist

- ☐ Cover page renders with correct building name and NEB class
- ☐ OBS percentage matches evaluation result
- ☐ Score distribution chart matches data
- ☐ All 63 criteria appear in the detail table
- ☐ Strengths and weaknesses lists are populated
- ☐ Page numbers appear on every page
- ☐ Table of contents has correct page references
- ☐ PDF is searchable (text, not images)
- ☐ PDF passes PDF/UA accessibility checker
- ☐ File size under 10MB

Automated Tests

```
        // tests/pdf-generation.test.ts
        describe('PDF Generation', () => {
          it('should generate a valid PDF buffer', async () => {
            const pdf = await generateReportPdf(testEvaluationId);
            expect(pdf).toBeInstanceOf(Buffer);
            expect(pdf.length).toBeGreaterThan(1000);
            // Check PDF magic bytes
            expect(pdf.slice(0, 5).toString()).toBe('%PDF-');
          });

          it('should include NEB classification in output', async
() => {
            const pdf = await generateReportPdf(testEvaluationId);
            const text = await pdfToText(pdf); // use pdf-parse
            expect(text).toContain('NEB CLASS');
            expect(text).toContain('Overall Building Score');
          });
        });
```

AASTool by Serg | Dev by Y